

A Zero-Knowledge Proof-of-Proof for a Clean-Room Certifying-Compiler Kernel

self-applied: the artifact certifies the proofs that produce it

2026-08-26

Abstract

This document is the public artifact of a zero-knowledge proof-of-proof (zkPoP) whose subject is the Lean 4 development that produced it. The development is a clean-room specification for certifying compilation: a transition-system semantics, simulation relations, and a `CertifyingCompiler` abstraction whose trusted decidable validator, once it accepts, yields a backward simulation and hence the CompCert-style guarantee that the shipped target exhibits no behaviour the source could not. On top of that sits a cryptography-free model of knowledge-sound argument systems, Pedersen commitments, and a Σ -protocol, and the theorems that weld them to the compiler-correctness statement. Here that machinery is turned on the development itself. A Lean development is a compilation too: the source is the corpus of proof scripts, the published target is the manifest of §2 — module names, digests of their text, and the theorem names they declare — and the validator recomputes the manifest from the corpus. Everything numeric below was computed by Lean from the actual files, including the file that proves the theorems being applied and the file that states the concrete claim.

1 The claim

A verifier holds two things and nothing else: the commitment C and the manifest M of §2. The prover keeps the corpus c — the source text of the proofs — and the commitment randomness r . The certified relation is

$$\text{selfRel}(C, M)(c, r) \equiv C = \text{commit}(\text{enc}(c), r) \wedge M = \text{manifestOf}(c),$$

and `manifestOf` is recomputation of every digest and every theorem list from the text. The claim, formally `project_zkpop_claim`, is:

There exists a proof corpus which opens the published commitment, whose recomputed manifest is exactly the published manifest, each of whose published entries is a genuine module of that corpus with that digest and those theorem names, and whose proof-checking behaviour the published manifest faithfully refines.

Verbatim, as it is stated and proved in `RequestProject/ZK/SelfManifest.lean`:

```
theorem project_zkpop_claim :  
  ∃ (c : Corpus) (r : ZMod q),  
    projectCommitment = commitEnc gGen hGen corpusScalar c r ∧  
    projectManifest = manifestOf c ∧
```

```

(∀ e ∈ projectManifest, ∃ m ∈ c, m.name = e.name ∧ fnv1a m.text = e.digest ∧
  m.theorems = e.theorems) ∧
(∀ b : Behavior String Unit,
  (manifestSem projectManifest).Behaves b → (corpusSem c).Behaves b) :=

```

The last conjunct is the compiler-correctness statement of the kernel, applied here. The observable semantics of a development (`checkingSem`) is its proof-checking transition system: one step per theorem checked, emitting that theorem’s name as an observable event. Refinement therefore says the manifest advertises no theorem the corpus does not check, in the order it checks them. Because the relation is an instance of `ccRel`, every theorem of the generic zkPoP layer applies verbatim: soundness for *any* knowledge-sound argument system, recursion to any depth, aggregation over a whole build, hiding, and binding.

2 The public artifact

modules covered	32
theorems covered	194
corpus digest	8bab860bf2b6b59
commitment C	515181
group	ZMod 1000003 (demonstration instance)
generators g, h	5, 7
digest function	fnv1a (FNV-1a, 64 bit; see §7)

The manifest itself follows. The digest column is the digest of the module’s full source text; the last column is the number of theorem names the module declares. §8 lists those names.

module	digest	thms
RequestProject.Automation	75f502cadb072214	0
RequestProject.CertifiedPass	b860e807b56a6414	6
RequestProject.CertifyingCompiler	368b82ea41eefc6	3
RequestProject.Contracts	45d64da422748dee	4
RequestProject.Datagram	a82e1ab1979d43ab	2
RequestProject.Examples.CompCertZK	c6831d02e919e40	9
RequestProject.Examples.ExprCertifying	ef10aebb37b4b509	10
RequestProject.Examples.ExprCompiler	80eb22a316c6cbc0	4
RequestProject.Examples.HeapSafety	61ff9ad6717031b	3
RequestProject.Examples.IRBundle	9f27b393a626b92	3
RequestProject.Examples.LoopInvariant	c1261d3e75a9b36	1
RequestProject.Examples.Optimize	a85dfff4880896e3	2
RequestProject.Examples.ProofOfProof	c73c0c226dbfe3c0	9
RequestProject.Examples.SelfCertifying	a00252e46830357b	1
RequestProject.Examples.StackMachine	bd2d0c8e9c007cf2	3
RequestProject.Heap	a516165adef2d065	7
RequestProject.IR	da10d3a6d47deab1	3
RequestProject.Main	79684de91510c995	0
RequestProject.Optimization	6a8402240adeab39	6
RequestProject.Simulation	a09c64289bfbaecf	6
RequestProject.Supersingular	23924953e9a356fd	7
RequestProject.SupersingularDatagram	c21ffaf4cf95655c	3

module	digest	thms
RequestProject.TransitionSystem	ec1145bb890cf964	5
RequestProject.TypeChecker	bf230150e6619c8b	3
RequestProject.ZK.ArgSystem	1e143aba9982735a	9
RequestProject.ZK.Commitment	909ef5ae5366cd42	9
RequestProject.ZK.CompCert	715f656869a66036	15
RequestProject.ZK.Encoding	9aa055776153e83e	10
RequestProject.ZK.ProofOfProof	454efc62bd805c91	12
RequestProject.ZK.SelfApply	622df9807e237d1f	23
RequestProject.ZK.SelfManifest	2e1ccbb0b9105779	10
RequestProject.ZK.Sigma	cb5e2ea29ed32255	6

3 What the verifier does

1. Read C and M from §2. Nothing else is needed, and nothing else is published by the protocol.
2. Run the argument system’s verifier on the claim (C, M) . Acceptance is the only input to the theorems below; the corpus is never inspected.
3. Conclude §1. Knowledge soundness turns acceptance into a corpus (`extract`), `zkpop_self_manifest` turns that into equality of the manifest with the recomputed one, and `validated_manifest_refines` turns it into the refinement.
4. Optionally recurse: verify a proof that step 2 was carried out, to any depth (`zkpop_self_recursive`), or verify one aggregated argument for many developments at once (`zkpop_self_build`).

The layer is deliberately parametric in the argument system: the theorems quantify over every complete, knowledge-sound system with an explicit extractor, so a succinct argument may be substituted for the transparent baseline `revealSelf` without touching a single proof. The baseline is what makes the claim demonstrably realizable rather than merely hypothetical (`revealSelf_honest`).

4 The machine-checked content

Each entry is a Lean theorem of the corpus certified above; the module column says where, and §8 shows it is covered by the manifest.

theorem	reading
<code>BackwardSimulation.refines</code> <code>Simulation</code>	A backward simulation makes the target refine the source: every behaviour of the target is a behaviour of the source.
<code>CertifyingCompiler.validated_refines</code> <code>CertifyingCompiler</code>	If the trusted validator accepts, the target exhibits no behaviour the source could not (the CompCert-style guarantee, per compilation).

theorem	reading
<code>CertifyingCompiler.preserves_safety</code> CertifyingCompiler	A validated compilation never turns a safe source into a target that goes wrong.
<code>tower_sound</code> ZK.ArgSystem	Recursive verification to any depth stays knowledge sound: a proof of a proof of a proof still yields a witness for the base relation.
<code>semantic_consequence_iff</code> ZK.ArgSystem	A property follows from acceptance exactly when it follows from satisfiability of the relation: the proof layer transports meaning, it does not create it.
<code>acceptance_alone_insufficient</code> ZK.ArgSystem	There is a complete, knowledge-sound, self-verifying system whose acceptance implies nothing: the content must come from the semantic theorems.
<code>ccRel_refines</code> ZK.CompCert	A witness for a compilation claim entails refinement of the committed source, which is never published.
<code>zkpop_claim</code> ZK.CompCert	For any knowledge-sound argument system, acceptance alone yields: there exists a source opening the commitment, validated against the shipped target, refined by it.
<code>validated_manifest_refines</code> ZK.SelfApply	A validated manifest refines the withheld corpus: it advertises exactly the theorem-checking behaviour of the proofs.
<code>zkpop_self_manifest</code> ZK.SelfApply	Acceptance implies the published manifest is the manifest recomputed from the extracted corpus.
<code>zkpop_self_claim</code> ZK.SelfApply	The self-applied claim: exactly what a verifier holding only the commitment and the manifest may conclude about the proofs it has not seen.
<code>zkpop_self_covers</code> ZK.SelfApply	For every published entry, the extracted corpus really contains a module with that name, that digest and those theorem names.
<code>zkpop_self_recursive</code> ZK.SelfApply	The self-applied conclusion survives recursion to any depth.
<code>zkpop_self_build</code> ZK.SelfApply	One aggregated argument certifies the manifest of a whole library of developments.
<code>selfApply_covers</code> ZK.SelfApply	Self-application: if the certified corpus contains a module — in particular the one proving these theorems — the published manifest carries its entry.
<code>self_hides_corpus</code> ZK.SelfApply	The published commitment is perfectly hiding: any other corpus is consistent with it after a bijective change of randomness.
<code>manifest_binds_corpus</code> ZK.SelfApply	Under an explicit collision-freeness hypothesis on the digest, the manifest determines the corpus.
<code>self_claim_binding</code> ZK.SelfApply	Under the discrete-logarithm assumption and an injective encoding, the commitment determines the corpus, so the claim cannot be repudiated.
<code>revealSelf_honest</code> ZK.SelfApply	The honest development produces an accepted claim, with no hypotheses: none of the above is vacuous.
<code>project_validated</code> ZK.SelfManifest	The trusted validator accepts this project's published manifest.
<code>project_refines</code> ZK.SelfManifest	This project's published manifest refines this project's withheld corpus.
<code>project_zkpop_claim</code> ZK.SelfManifest	The self-applied claim, instantiated at this development and its published artifact.
<code>project_covers_selfApply</code> ZK.SelfManifest	The certified corpus contains the module that proves the self-applied theorems.
<code>project_covers_selfManifest</code> ZK.SelfManifest	It also contains the module carrying the concrete claim, so the artifact certifies its own manifest.

theorem	reading
<code>project_recursive_depth_three</code> <code>ZK.SelfManifest</code>	A proof of a proof of a proof of this project’s manifest.

5 Self-application

The corpus is not a description of the development from outside: it is built with `include_str` from the files themselves, so the two modules that carry the argument are elements of the corpus they certify.

module	digest	theorems
<code>RequestProject.ZK.SelfApply</code>	622df9807e237d1f	23
<code>RequestProject.ZK.SelfManifest</code>	2e1ccbb0b9105779	10

Consequently `project_covers_selfApply` and `project_covers_selfManifest` are theorems: the entry of the module that proves `zkpop_self_claim`, and the entry of the module that states the concrete claim, both belong to the published manifest. The claim of §1 is therefore a claim about a corpus which contains its own justification — the digests in §2 were computed by Lean from those very files at build time, so the document cannot drift from what it describes. Iterating the argument gives `project_recursive_depth_three`: a proof of a proof of a proof of this manifest, with the same conclusion.

Two limits keep this honest rather than circular. `acceptance_alone_insufficient` exhibits a complete, knowledge-sound, self-verifying system whose acceptance implies nothing whatsoever, and `semantic_consequence_iff` shows a proof layer can only transport the meaning already present in its base relation. A self-applied zkPoP therefore certifies that the corpus is the one that was committed to and that it declares the advertised theorems; that those theorems are true is a matter for the Lean kernel, whose verdict is recorded next.

6 Kernel verdict

The development builds with `lake build`. A scan of the corpus for `sorry`, `admit`, `axiom` and `@[implemented_by]` reports no occurrences. The axiom closure of the headline theorems, as printed by Lean, is below; every entry is contained in Lean’s standard foundational axioms `propext`, `Classical.choice`, `Quot.sound`. No cryptographic hardness assumption is an axiom anywhere: hiding and binding take theirs as explicit hypotheses of the statements that use them.

theorem	axioms
<code>CertCompiler.BackwardSimulation.refines</code>	<code>propext</code>
<code>CertCompiler.CertifyingCompiler.validated_refines</code>	<code>propext</code>
<code>CertCompiler.CertifyingCompiler.preserves_safety</code>	<code>propext</code>
<code>CertCompiler.ZK.ccRel_refines</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.zkpop_claim</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.zkpop_recursive</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.zkpop_build</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>

theorem	axioms
<code>CertCompiler.ZK.SelfApply.validated_manifest_refines</code>	<code>propext</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.zkpop_self_manifest</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.zkpop_self_claim</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.zkpop_self_recursive</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.zkpop_self_build</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.zkpop_self_covers</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.selfApply_covers</code>	<code>propext</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.manifest_binds_corpus</code>	<code>propext</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.self_hides_corpus</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.self_claim_binding</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.revealSelf_honest</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.Project.project_validated</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.Project.project_refines</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.Project.project_accepted</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.Project.project_zkpop_claim</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.Project.project_covers_</code> <code>selfApply</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.Project.project_covers_</code> <code>selfManifest</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.Project.project_zkpop_covers</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
<code>CertCompiler.ZK.SelfApply.Project.project_recursive_</code> <code>depth_three</code>	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>

Axioms observed overall: `Classical.choice`, `Quot.sound`, `propext`.

7 Scope and trust assumptions

- `fnv1a` is a fast, kernel-computable *demonstration* digest, not a cryptographic hash. Nothing is claimed about its collision-resistance: the digest-level binding theorem `manifest_binds_corpus` takes injectivity as an explicit hypothesis, exactly as the commitment-level result takes the discrete-logarithm assumption. Replacing the digest by a real hash changes no proof.
- The commitment lives in `ZMod 1000003`, a toy group chosen so that the value in §2 is computed by the kernel. The commitment theorems are proved for an arbitrary `AddCommGroup` that is a module over `ZMod q`; only the displayed number is toy.
- This repository publishes its sources, and the commitment randomness is a fixed constant in the generated module. The artifact therefore *exhibits* the protocol; it does not actually conceal a corpus. The hiding theorem `self_hides_corpus` is what would do so in a deployment.
- The manifest binds text, not meaning. A digest certifies which bytes were committed to; that those bytes constitute proofs is established by the Lean kernel, and the axiom table

above is that report. A deployment that wants the kernel’s verdict itself inside the zero-knowledge statement must arithmetize the type-checker — the layer is already parametric over the argument system that would do it.

- The theorem list of a module is read out of its source text by `declaredTheorems`, a line-based reader for declarations beginning with `theorem` or `lemma`, optionally after an attribute. It is derived from the text rather than asserted beside it, but it is a lexical reader, not the Lean parser.

8 Index of certified theorems

All 194 theorem names carried by the manifest, by module.

```
RequestProject.CertifiedPass
  foldInstr_exec, foldlM_exec_map, constFold_correct, CertifiedPass.validate_sound, Bundle.sound,
  Bundle.preserves_behavior
RequestProject.CertifyingCompiler
  CertifyingCompiler.validated_refines, CertifyingCompiler.compiled_refines, CertifyingCompiler.
  preserves_safety
RequestProject.Contracts
  Contract.Verified.inv_star, Contract.Verified.partial_correct, Contract.Verified.terminates_
  post, Contract.Verified.safe_of_progress
RequestProject.Datagram
  Datagram.partial_correct, Datagram.safe
RequestProject.Examples.CompCertZK
  zk_claim_accepted, zk_shipped_refines, zk_shipped_never_goes_wrong, zk_verifier_view, zk_
  shipped_computes_secret, zk_hides_source, zk_claim_binding, zk_tower_accepted, zk_tower
RequestProject.Examples.ExprCertifying
  trace_nil, star_inv, star_cases, star_of_empty_code, step_det, exprCC_self_validates, exprCC_
  refines, exprSem_safe, exprCC_preserves_safety, exprCode_injective
RequestProject.Examples.ExprCompiler
  compile_correct, exprSem_terminates, compile_preserves_eval, example_compile
RequestProject.Examples.HeapSafety
  arraySum_safe, write_frames, write_preserves_dom
RequestProject.Examples.IRBundle
  bundle_sound, fold_happened, bundle_preserves_behavior
RequestProject.Examples.LoopInvariant
  loop_partial_correct
RequestProject.Examples.Optimize
  optimized_partial_correct, dead_branch_removed
RequestProject.Examples.ProofOfProof
  demo_accepted, demo_semantics, demo_behaves, demo_opens, sigma_demo_accepted, sigma_demo_
  extracts, sigma_demo_simulatable, tower_demo_accepted, tower_demo
RequestProject.Examples.SelfCertifying
  fixedObligations_wellTyped
RequestProject.Examples.StackMachine
  example_terminates, idCompiler_refines, foldSim_refines
RequestProject.Heap
  read_empty, read_write_same, read_write_ne, read_write_frame, dom_write_same, dom_write, read_
  write_get
RequestProject.IR
  progSem_terminates, progSem_star, progSem_behaves_iff
RequestProject.Optimization
  Contract.deadCodeElim_step_sub, Contract.Verified.deadCodeElim, Contract.deadCodeElim_star_
  sound, Contract.Verified.deadCodeElim_star_complete, Contract.deadCodeElim_preserves_
  terminates, Datagram.deadCodeElim_partial_correct
```

```

RequestProject.Simulation
  ForwardSimulation.match_star, ForwardSimulation.preserves_terminates, BackwardSimulation.
  match_star, BackwardSimulation.preserves_terminates, BackwardSimulation.preserves_goesWrong,
  BackwardSimulation.refines

RequestProject.Supersingular
  monster_factorization_prod, supersingularPrimes_length, supersingularPrimes_prime,
  supersingularPrimes_eq_factorization_primes, userDimensions_prod, userDimensions_primes,
  monsterOrder_eq_userDimensions_mul

RequestProject.SupersingularDatagram
  totalDimension_eq, levelPrime_prime, GradedDatagram.partial_correct

RequestProject.TransitionSystem
  Star.single, Star.trans, Plus.star, Plus.single, Semantics.Safe.not_goesWrong

RequestProject.TypeChecker
  Obligation.check_iff, feedback_eq_nil_iff, WellTyped.all_hold

RequestProject.ZK.ArgSystem
  exists_witness, recurse_extract, recurse_accepts, tower_sound, tower_complete, recurse_sound,
  semantic_consequence_iff, vacuous_iff_rel_total, acceptance_alone_insufficient

RequestProject.ZK.Commitment
  commit_add, commit_smul, commit_perfectly_hiding, commit_binding_extract_dlog, commit_binding,
  commit_binding_message, commitProgram_hiding, commit_binds_encoding, commit_binds_program

RequestProject.ZK.CompCert
  ccRel_refines, ccRel_preserves_safety, ccRel_opens, ccRel_simulation, zkpop_refines, zkpop_
  preserves_safety, zkpop_opens, zkpop_claim, zkpop_recursive, zkpop_build, revealCC_honest,
  revealCC_guarantee, cc_hides_source, cc_claim_binding, cc_transcripts_simulatable

RequestProject.ZK.Encoding
  strCode_injective, opCode_injective, operandCode_injective, instrCode_injective, programCode_
  injective, encodeProgram_injective, progField_injective, progField_inj, validateSat_iff,
  validateSat_sound

RequestProject.ZK.ProofOfProof
  committedRel_sound, committedRel_behaves, committedRel_opens, end_to_end, end_to_end_behaves,
  end_to_end_opens, end_to_end_recursive, end_to_end_build, revealArg_honest, artifact_hides_
  source, claim_binding, proof_of_proof

RequestProject.ZK.SelfApply
  checkManifest_eq_true, entryOf_theorems, entryOf_name, mem_manifestOf_iff, entryThms_
  manifestOf, manifestSem_eq_corpusSem, leanCC_check, leanCC_self_check, validated_manifest_
  refines, selfRel_manifest, selfRel_opens, selfRel_refines, zkpop_self_manifest, zkpop_self_
  claim, zkpop_self_recursive, zkpop_self_build, revealSelf_honest, self_hides_corpus, self_claim_
  binding, entryOf_injective, manifest_binds_corpus, selfApply_covers, zkpop_self_covers

RequestProject.ZK.SelfManifest
  project_validated, project_check_accepts, project_witness_valid, project_refines, project_
  accepted, project_zkpop_claim, project_covers_selfApply, project_covers_selfManifest, project_
  zkpop_covers, project_recursive_depth_three

RequestProject.ZK.Sigma
  completeness, special_soundness, simulator_verifies, real_eq_sim, accepting_iff_sim, hvzk

```

9 Reproducing this document

1. `lake build` — build the development.
2. `python3 scripts/gen_self_manifest.py` — regenerate `RequestProject/ZK/SelfManifest.lean` from the files on disk.
3. `lake env lean scripts/AxiomCheck.lean` — print the axiom closures.
4. `python3 scripts/gen_zkpop_pdf.py` — rebuild and re-elaborate the manifest module, collect the numbers above and typeset this document. The rebuild is forced, because `include_str` dependencies are not part of the build system's traces and a stale artifact would report the digests of an older state of the files.

Step 4 recomputes every digest from the current files, so a single changed byte anywhere in the corpus changes the manifest, the corpus digest and the commitment.